Numbers, Variables and Units

Engineering Computation with Python — Chapter 3

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Beyond Simple Numbers
2. Collections
3. Mutability
4. Engineering Units
5. Conventions and Constants
6. Summary

© Muním Zabidi

Table of Contents

1. Beyond Simple Numbers
2. Collections
3. Mutability
4. Engineering Units
5. Conventions and Constants
6. Summary

Engineering Data Is Often More Complex Than This

Chapter 2 covered values that hold a single number or word. Engineering data is often more complex:

- An AC voltage needs two components at once.
- A measurement run produces a whole series of readings.
- Every quantity carries a unit that decides whether the answer is right.

This chapter

Richer types for all three — complex numbers, strings, and four collection types — then how to represent quantities like 4.7 kΩ and 10 μF correctly.

© Muním Zabidi

Complex Numbers in Python

Python natively supports complex numbers: $a + bj$, where j is the imaginary unit ($j^2 = -1$).

```
c1 = 3 + 4j   # real part 3, imaginary part 4
c2 = -2 - 5j
print(c1)          # (3+4j)
print(type(c1))    # <class 'complex'>

c = complex(3, 4)  # equivalent to 3 + 4j
```

Python uses `j`, not the mathematician's `i` — matching electrical-engineering convention, where i already means current.

Working with Complex Numbers

```python
c = 3 + 4j
print(c.real)  # 3.0
print(c.imag)  # 4.0

c1 = 3 + 4j
c2 = 1 - 2j
print(c1 + c2)  # (4+2j)
print(c1 * c2)  # (11-2j)
```

```python
c = 3 + 4j
print(c.conjugate())
# (3-4j)

print(abs(c))
# 5.0  (magnitude)
```

The four arithmetic operators work exactly as on real numbers — Python handles the real/imaginary parts for you.

Why Complex Numbers Matter in EE

Signal processing and electrical engineering

Commonly used for oscillations or waves — AC circuit analysis represents voltage and current as complex phasors, making impedance calculations straightforward.

Mathematics

Useful for solving equations involving square roots of negative numbers.

Complex numbers for AC circuits return in full in Chapter 8.

© Munim Zabidi

Strings: Text as a Sequence

A string is a sequence of characters enclosed in quotes. Strings are immutable — they cannot be changed after creation.

```
string1 = 'Hello'
string2 = "World"
string3 = '''A multi-line
string.'''
```

Single and double quotes are interchangeable; triple quotes span several lines.

© Muním Zabidi

Indexing and Slicing Strings

```
text = "Python"
first_char = text[0]   # 'P'
last_char = text[-1]   # 'n'

text = "Python Programming"
slice1 = text[0:6]  # 'Python'
slice2 = text[7:]   # 'Programming'
```

Remember

Counting starts from zero. A negative index counts back from the end. `stop` in a slice is excluded.

String Methods

Because strings are immutable, every method below returns a new string rather than altering the original.

```
text = " Python Programming "

text.upper()     # ' PYTHON PROGRAMMING '
text.lower()     # ' python programming '
text.strip()     # 'Python Programming'
text.replace("Python", "Java")
# ' Java Programming '
text.split()     # ['Python', 'Programming']
```

Table of Contents

1 Beyond Simple Numbers

2 Collections

3 Mutability

4 Engineering Units

5 Conventions and Constants

6 Summary

Four Ways to Hold Several Values

So far every variable has held exactly one value. Engineering work rarely stays that simple: a measurement run produces a series of readings, a circuit has a set of component values.

List	Tuple	Set	Dictionary
Ordered, changeable	Ordered, fixed	Unordered, unique	Looked up by name

Choosing the wrong one is a common source of awkward code.

Lists: The Everyday Workhorse

```
readings = [26.5, 26.6, 26.5, 26.7, 26.6]
print(readings[0])     # 26.5
print(readings[-1])    # 26.6
print(len(readings))   # 5

readings.append(27.0)  # add to the end
readings[1] = 99.9     # lists are mutable
```

Ordered and changeable

A list keeps items in the order you put them, and you can add, remove, or reassign elements after creation.

Slicing a List

```
data = [10, 20, 30, 40, 50, 60]

print(data[1:4])    # [20, 30, 40]
print(data[:3])     # [10, 20, 30]
print(data[3:])     # [40, 50, 60]
print(data[::2])    # [10, 30, 50]  -- every 2nd
print(data[::-1])   # reversed
```

`[start:stop:step]` — extract a window, a tail, or every *n*th element. `stop` is always excluded.

Tuples: Values That Belong Together

```
point = (3, 4)
rgb_color = (255, 128, 0)

red, green, blue = (255, 128, 0)  # unpacking
print(red, green, blue)  # 255 128 0

# point[0] = 5  # TypeError -- tuples are immutable
```

Immutable by design

The right choice when values belong together permanently and should not be edited by accident: an (x, y) coordinate, an RGB colour, a fixed pair of circuit terminals.

Tuples Reappear: NumPy Shapes

You will meet tuples again sooner than you might expect.

The shape of a NumPy array is a tuple

`np.zeros((3, 3))` — the dimensions are passed inside an extra pair of brackets, because the shape itself is a single tuple argument.

This becomes essential terminology from Chapter 6 onward.

Sets: Only Distinct Values Matter

A set is an unordered collection that automatically discards duplicates.

```
readings = [26.5, 26.6, 26.5, 26.7, 26.6]
unique = set(readings)
print(sorted(unique))   # [26.5, 26.6, 26.7]
print(len(unique))      # 3 distinct values

a = {1, 2, 3}
b = {3, 4}
print(a | b)   # union: {1, 2, 3, 4}
print(a & b)   # intersection: {3}
```

No order means no indexing — `unique[0]` is an error.

Dictionaries: Looked Up by Name

A dictionary stores values looked up by a key rather than a numeric position — `key: value` pairs.

```
components = {"R1": 4700, "R2": 100, "C1": 100e-6}

print(components["R1"])     # 4700
components["R3"] = 220      # add a new entry

for name, value in components.items():
    print(f"{name}: {value}")
```

A list would force you to remember that position 0 means R1 — a fact nothing in the code records.

Choosing a Collection

Type	Written	Ordered?	Mutable?	Use it when
list	`[1, 2, 3]`	yes	yes	values form a changing sequence
tuple	`(1, 2, 3)`	yes	no	values belong together, fixed
set	`{1, 2, 3}`	no	yes	only distinct values matter
dict	`{"a": 1}`	yes	yes	values are looked up by name

For numerical work

The list is by far the most important — and it is itself soon superseded by the NumPy array in Chapter 6.

© Muním Zabidi

Table of Contents

1. Beyond Simple Numbers
2. Collections
3. **Mutability**
4. Engineering Units
5. Conventions and Constants
6. Summary

Mutable vs. Immutable

Mutable: can change in place

list, set, dictionary

```
a = [1, 2, 3]
b = a          # b refers to
               # the SAME list
b.append(4)
print(a)
# [1, 2, 3, 4]  -- a changed too!
```

Immutable: cannot change

int, float, str, tuple, complex, bool

```
x = 5
y = x
y = y + 1
print(x)
# 5  -- x is unaffected
```

Testing Immutability with `id()`

`id()` returns a variable's memory address — the same tool from Chapter 2, now applied to lists and tuples:

```
readings = [26.5, 26.6]
before = id(readings)
readings.append(26.7)
print(id(readings) == before)   # True -- same object

point = (2.5, 4.0)
before = id(point)
point = point + (1.0,)
print(id(point) == before)      # False -- new object
```

Appending to a list edits it in place; "adding" to a tuple cannot, so Python builds a new one.

© Muním Zabidi

The hazard

Assigning a mutable object to a new name does not copy it — both names point to the same underlying data. A change through one name is visible through the other.

Practical consequence

Passing a list into a function and modifying it inside the function changes the *caller's* list too, unless you deliberately make a copy first. This surprises nearly every beginner at least once.

© Munim Zabidi

Table of Contents

1 Beyond Simple Numbers

2 Collections

3 Mutability

4 **Engineering Units**

5 Conventions and Constants

6 Summary

Every Number Carries a Unit

A resistance of 470 is meaningless; a resistance of 470 Ω is a component you can hold.

Python does not track units

It will happily add a voltage to a resistance and give you a number. Keeping track of units is your responsibility — one of the most common sources of error for beginners.

Quantities You Will Meet Most

Quantity	Sym.	Unit	Relationship
Voltage	V	volt (V)	$V = IR$
Current	I	ampere (A)	$I = V/R$
Resistance	R	ohm (Ω)	$R = V/I$
Power	P	watt (W)	$P = VI$
Capacitance	C	farad (F)	$i = C\,dv/dt$
Frequency	f	hertz (Hz)	$f = 1/T$

SI Prefixes You Will Use Constantly

A capacitor might be 10 microfarads while a resistor is 4.7 kilohms — writing either in full decimal form is error-prone.

Prefix	Sym.	Mult.	Example
kilo	k	10^{3}	4.7 kΩ
milli	m	10^{-3}	20 mA
micro	μ	10^{-6}	10 μF
nano	n	10^{-9}	5 ns
pico	p	10^{-12}	22 pF

© Munim Zabidi

Scientific Notation in Python

Python has no notion of “kilo” or “micro” — prefixed values must be entered as plain numbers, using scientific notation.

```
R = 4.7e3      # 4.7 kilohms = 4700 ohms
C = 100e-6     # 100 microfarads = 0.0001 farads
I = 20e-3      # 20 milliamps = 0.02 amps
```

The letter e means “times ten to the power of.” This is the clearest way to write a prefixed engineering value.

Working Consistently in Base Units

The rule

Convert every quantity to its base SI unit (ohms, farads, seconds, amps) before calculating, then convert the *result* back to a convenient prefix for display.

Worked check: RC time constant

$R = 4.7\,\text{k}\Omega = 4700\,\Omega$, $C = 100\,\mu\text{F} = 0.0001\,\text{F}$:

$$\tau = RC = 4700 \times 0.0001 = 0.47\,\text{s}.$$

Units confirm the result: ohms $\times$ farads $=$ seconds.

Table of Contents

1 Beyond Simple Numbers

2 Collections

3 Mutability

4 Engineering Units

5 **Conventions and Constants**

6 Summary

Naming Conventions: PEP 8

Python does not care what you call things — `supply_voltage`, `SupplyVoltage`, and `sv` are all legal. Other engineers do care.

snake_case	SCREAMING_SNAKE	CapWords
variables, functions `supply_voltage`	constants `MAX_CURRENT`	classes `Resistor`

The style you will use constantly is snake_case.

Naming Well: A Practical Example

```python
# Conventional -- every name announces what it is
MAX_CURRENT = 0.02              # a constant
supply_voltage = 5.0            # a variable

def series_resistor(v_supply, v_led, current):
    return (v_supply - v_led) / current
```

Two further habits

Name the *quantity*, not its type (`voltage` beats `voltage_float`). Reserve single letters for symbols that match the mathematics (`R`, `C`, `f`, `t`).

Some quantities never change: the speed of light, a supply rail fixed by hardware. Python has no `const` keyword — it uses the `SCREAMING_SNAKE_CASE` convention instead.

```
PI = 3.14159
GRAVITY = 9.81
SPEED_OF_LIGHT = 299792458
```

The capitals carry no authority

Nothing stops a later line from reassigning `GRAVITY` — a constant in Python is a promise between programmers, not a guarantee from the language.

© Muním Zabidi

Table of Contents

1 Beyond Simple Numbers

2 Collections

3 Mutability

4 Engineering Units

5 Conventions and Constants

6 **Summary**

What You Should Take Away

- **Complex numbers** (`3+4j`) and **strings** are Python's richer scalar types, both with their own arithmetic and methods.
- Four **collections**: list (ordered, mutable), tuple (ordered, fixed), set (unordered, unique), dictionary (keyed lookup).
- **Mutability** matters: assigning a mutable object to a new name shares the same underlying data.
- Every number carries a **unit**. Work in **base SI units**, using scientific notation (`4.7e3`) for prefixed values.

© Muním Zabidi

What You Should Take Away (continued)

- **PEP 8** naming: `snake_case` for variables and functions (your everyday style), `SCREAMING_SNAKE_CASE` for constants.
- A Python **constant** is a naming convention, not an enforced guarantee — honouring it is what makes it useful.
- The **list** is the most important collection for numerical work — and is superseded by the NumPy **array** from Chapter 6 onward.

Next: Decisions and Repetition

© Muním Zabidi